

Presentación del Backend de Microservicios: Gestión de Productos e Inventario

1. Introducción

Este documento presenta la arquitectura y el flujo de datos de un sistema de gestión de productos e inventario implementado con microservicios en .NET, bases de datos SQL Server y orquestado con Docker Compose. El objetivo principal es demostrar un diseño escalable, resiliente y mantenible.

2. Arquitectura General del Sistema

El sistema se compone de varios servicios independientes que interactúan entre sí, cada uno con su propia responsabilidad y base de datos.

graph TD

subgraph Cliente/Frontend

A[Navegador/App Móvil] --> B[API Gateway / Load Balancer]

end

subgraph Red Docker Compose

B --> C[ProductsService]

B --> D[InventoryService]

C --> E[SQL Server (ProductsDb)]

D --> F[SQL Server (InventoryDb)]

D --> C

end

style A fill:#f9f,stroke:#333,stroke-width:2px

style B fill:#bbf,stroke:#333,stroke-width:2px

style C fill:#ccf,stroke:#333,stroke-width:2px

style D fill:#ccf,stroke:#333,stroke-width:2px

```
style E fill:#fcc,stroke:#333,stroke-width:2px
```

```
style F fill:#fcc,stroke:#333,stroke-width:2px
```

```
linkStyle 0 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;
```

```
linkStyle 1 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;
```

```
linkStyle 2 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;
```

```
linkStyle 3 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;
```

```
linkStyle 4 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;
```

```
linkStyle 5 stroke:#00f,stroke-width:2px,fill:none,stroke-dasharray: 5 5;
```

```
subgraph Leyenda
```

```
  L1[Caja Blanca: Contenedor/Servicio]
```

```
  L2[Caja Roja: Base de Datos]
```

```
  L3[Flecha Azul Discontinua: Llamada HTTP/API]
```

```
  L4[Flecha Negra Sólida: Conexión Directa a DB]
```

```
end
```

Componentes Principales:

- **Cliente/Frontend (Angular):** La interfaz de usuario que interactúa con los microservicios a través de solicitudes HTTP.
- **ProductsService:** Microservicio dedicado a la gestión de la información de los productos (nombre, descripción, precio). Persiste sus datos en **ProductsDb**.
- **InventoryService:** Microservicio encargado de la gestión de las cantidades de stock de los productos. Persiste sus datos en **InventoryDb**.
- **SQL Server (ProductsDb / InventoryDb):** Bases de datos relacionales separadas para cada microservicio, garantizando la autonomía de los datos.
- **Docker y Docker Compose:** Herramientas para contenerizar y orquestar todos los servicios, asegurando un entorno de ejecución consistente y portátil.

3. Flujo de Datos (Ejemplo: Actualización de Inventario)

Para ilustrar la interacción entre los servicios, consideremos el flujo cuando un usuario actualiza la cantidad de inventario de un producto:

sequenceDiagram

participant C as Cliente/Frontend

participant IS as InventoryService

participant PS as ProductsService

participant IDB as InventoryDb

participant PDB as ProductsDb

C->>IS: 1. PATCH /inventories/{productId} (Actualizar Cantidad)

IS->>PS: 2. HTTP GET /products/{productId} (Verificar Existencia y Detalles del Producto)

PS->>PDB: 3. Consulta Producto por ID

PDB-->>PS: 4. Detalles del Producto

PS-->>IS: 5. 200 OK (Detalles del Producto)

IS->>IDB: 6. Consulta InventoryItem por ProductId

alt Si no existe en InventoryDb

IDB-->>IS: 7. No encontrado

IS->>IDB: 8. Inicializa InventoryItem (Quantity: 0)

IDB-->>IS: 9. InventoryItem Creado

end

IS->>IDB: 10. Actualiza Quantity y LastUpdated

IDB-->>IS: 11. Cambios Guardados

IS-->>C: 12. 200 OK (Inventario Actualizado)

Explicación del Flujo:

1. **Solicitud de Actualización:** El cliente envía una solicitud **PATCH** al **InventoryService** para cambiar la cantidad de un producto.

2. **Verificación Inter-Servicio:** Antes de modificar su propio inventario, el `InventoryService` realiza una llamada HTTP (GET) al `ProductsService` para **verificar que el `productId` exista** y obtener los detalles del producto. Esta es una comunicación entre servicios.
3. **Consulta de Inventario:** El `InventoryService` consulta su propia base de datos (`InventoryDb`) para el `InventoryItem` asociado al `productId`. Si no existe, lo inicializa con cantidad cero.
4. **Actualización y Persistencia:** El `InventoryService` calcula la nueva cantidad, actualiza el `InventoryItem` y persiste los cambios en `InventoryDb`.
5. **Respuesta al Cliente:** Finalmente, el `InventoryService` envía una respuesta `200 OK` al cliente con los detalles del inventario actualizado.

Este flujo destaca la **separación de responsabilidades** y la **comunicación basada en API** entre microservicios, sin que un servicio acceda directamente a la base de datos de otro.

4. Decisiones Técnicas Clave

El diseño de este sistema se basa en varias decisiones técnicas fundamentales para asegurar su robustez y escalabilidad:

- **Arquitectura de Microservicios:** Permite el escalado, despliegue y desarrollo independiente de cada componente, mejorando la agilidad y resiliencia.
- **.NET 8:** Elegido por su alto rendimiento, soporte multiplataforma y un ecosistema maduro que acelera el desarrollo.
- **Docker y Docker Compose:** Facilitan la contenerización y orquestación de los servicios y la base de datos, garantizando consistencia ambiental y portabilidad.
- **SQL Server con Entity Framework Core:** Proporciona una base de datos relacional fiable para la persistencia de datos transaccionales, con un ORM que aumenta la productividad del desarrollador y gestiona las migraciones de esquema.
- **JSON:API:** Una especificación estándar para la comunicación API que asegura consistencia, eficiencia y una mejor experiencia para los desarrolladores que consumen la API.
- **GUIDs para Identificadores:** Garantiza la unicidad global de los IDs y permite la generación descentralizada de identificadores en un entorno distribuido.
- **Patrones de Resiliencia (Polly y `EnableRetryOnFailure`):** Implementados para manejar fallos transitorios (ej., problemas de red, reinicios de DB) mediante reintentos automáticos y circuitos de interrupción, mejorando la estabilidad del sistema.
- **Bases de Datos Separadas por Servicio:** Cada microservicio posee su propia base de datos, lo que refuerza el desacoplamiento, la autonomía y la capacidad de evolución independiente de los esquemas de datos.

5. Conclusión

Este backend de microservicios representa una solución moderna y robusta para la gestión de productos e inventario. Su diseño modular y el uso de tecnologías probadas garantizan

un sistema escalable, fácil de mantener y resiliente frente a fallos, preparado para crecer y adaptarse a futuras necesidades del negocio.